

16. Sicherheit

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

Vorschau

- Bedrohungen & Angriffsarten
- Fallbeispiel: ARP Spoofing - Probleme
- Sicherheitsmechanismen
- Symmetrische Verschlüsselung
- Asymmetrische Verschlüsselung
- Digitale Signatur
- Vergleich der Verschlüsselungsverfahren
- Fallbeispiel: ARP Spoofing - Lösungen

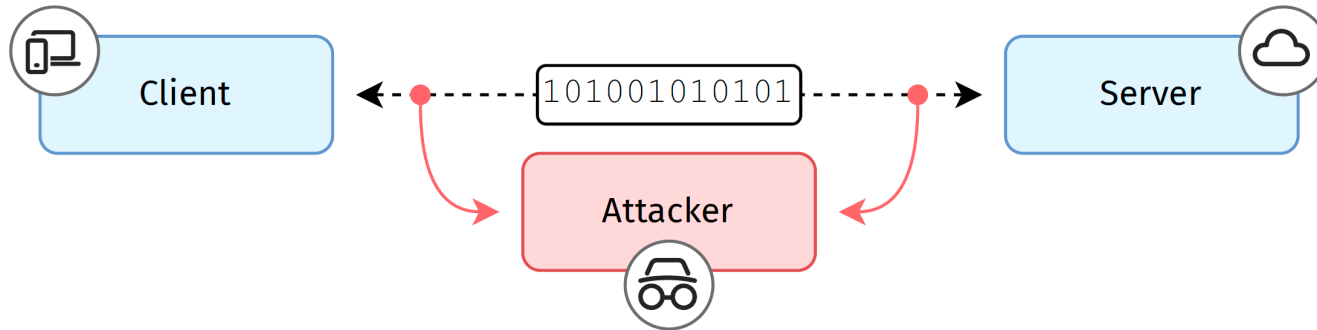


16.1 Bedrohungen & Angriffsarten



Grundlagen

- In einem verteilten System findet immer eine Kommunikation über ein Netzwerk statt.
- Ohne Gegenmaßnahmen kann sämtliche Kommunikation mitgelesen und/oder manipuliert werden



Wer kann angreifen?

- Eine angestellte Person im firmeninternen Netzwerk
- Der eigene Internetprovider, über welchen die gesamte Kommunikation läuft
- Eine Person, welche einen Fake-Hotspot (Wi-Fi) öffnet
- Ein Geheimdienst, welcher Knotenpunkte im Internet kontrolliert
- ...



Bedrohungen

- Engl. threats
- Unbefugter Zugriff auf Informationen und Dienste
- Beeinträchtigung der Leistungsfähigkeit eines Systems
- Unbefugtes Einbringen neuer Daten oder neuer „Dienste“, ...



Angriffsarten

- **Denial of Service Attacke**: Versuch einen Dienst unbrauchbar zu machen
- **Lauschangriff**: unbefugtes Abhören einer Kommunikation
- **Maskerade**: Vorgeben einer falschen Identität
- **Infiltration**: eigene Komponenten werden eingeschleust
 - Virus: angehängt an legale Daten; wird unbeabsichtigt aktiviert; repliziert sich, ...
 - Wurm: unbefugtes Einbringen und Ausführen von Programmen (Sicherheitslöcher)
 - Trojanisches Pferd: angeblich legitime Programme entpuppen sich als Schädling.

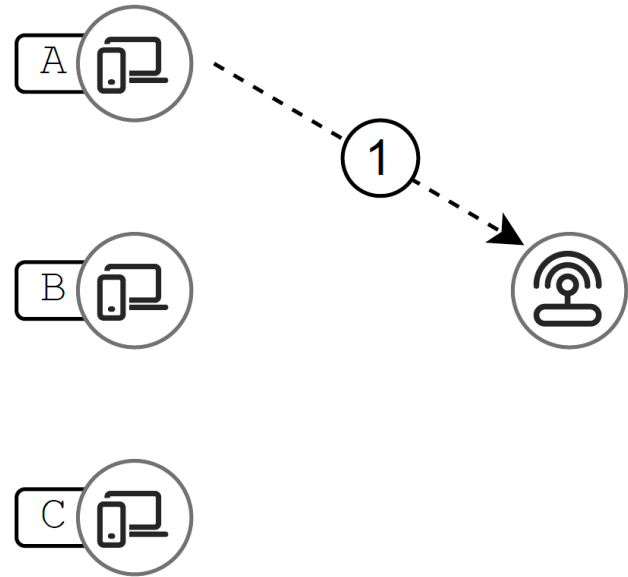


16.2 Fallbeispiel: ARP Spoofing - Probleme



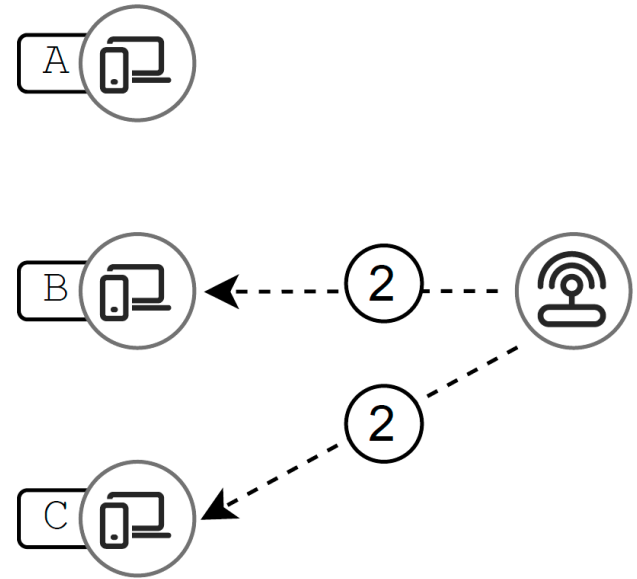
ARP-Protokoll

- Innerhalb eines Local Area Networks (LAN) wird das Address Resolution Protocols (ARP) verwendet, um zu einer IP-Adresse die zugehörige MAC-Adresse zu suchen.
- Ablauf:
 1. Host A schickt ARP-Nachricht an den Gateway (Router)



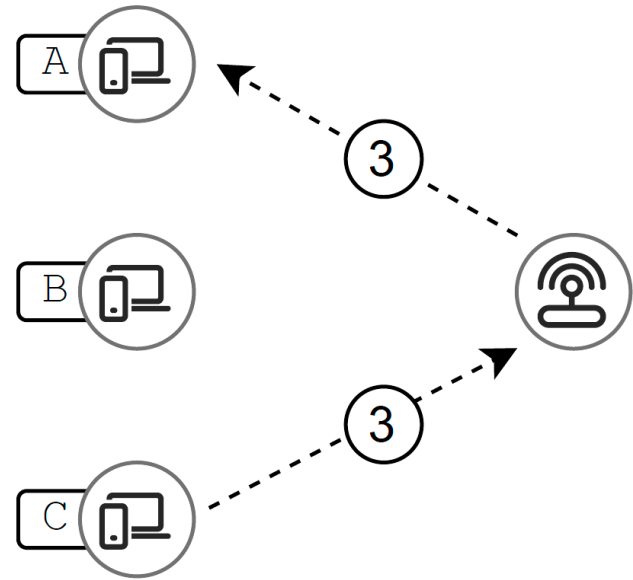
ARP-Protokoll

- Innerhalb eines Local Area Networks (LAN) wird das Address Resolution Protocols (ARP) verwendet, um zu einer IP-Adresse die zugehörige MAC-Adresse zu suchen.
- Ablauf:
 1. Host A schickt ARP-Nachricht an den Gateway (Router)
 2. Gateway leitet ARP-Nachricht an **alle** Hosts weiter



ARP-Protokoll

- Innerhalb eines Local Area Networks (LAN) wird das Address Resolution Protocols (ARP) verwendet, um zu einer IP-Adresse die zugehörige MAC-Adresse zu suchen.
- Ablauf:
 1. Host A schickt ARP-Nachricht an den Gateway (Router)
 2. Gateway leitet ARP-Nachricht an **alle** Hosts weiter
 3. Host C besitzt die richtige IP-Adresse und antwortet



- Eine Anfrage innerhalb des ARP-Protokolls beinhaltet unter anderem...
 - die IP-Adresse der Quelle
 - die MAC-Adresse der Quelle
 - die IP-Adresse des Ziels
- Erreicht die ARP-Anfrage das richtige Ziel (IP-Adresse stimmt überein), so antwortet das Ziel dem anfragenden Client mit seiner MAC-Adresse
⇒ MAC-Adresse des Ziels wird in der ARP-Tabelle des Clients gecached
- Da die Suche per Broadcast realisiert ist, wird das Ergebnis gecacht
⇒ Cache-Einträge sind je nach Konfiguration 1 – 2min. gültig

ARP Spoofing

- Problem: Das ARP-Protokoll arbeitet ohne jegliche Authentifizierung
⇒ Nach einer Anfrage wird die erste darauf folgende Antwort genutzt, um die MAC-Adresse innerhalb der ARP-Tabelle zu aktualisieren
- Welche Konsequenzen sind damit verbunden?
- Jeder beliebige Computer im LAN kann falsche Antworten versenden!



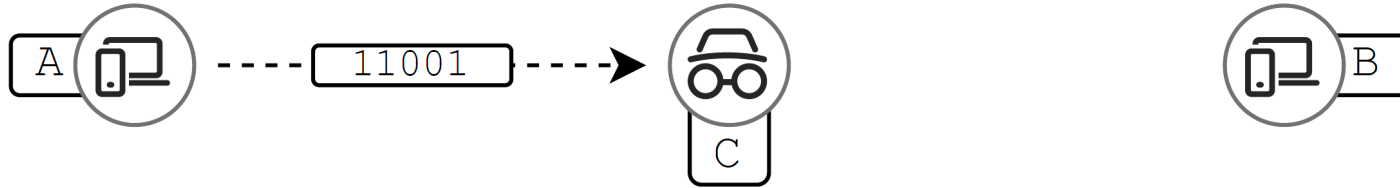
ARP Spoofing

- Der Angreifer schickt kontinuierlich ARP-Antworten an das Angriffsziel und wartet, bis dieses die "falsche" MAC-Adresse eingetragen hat.
- Möglichkeiten nach einem erfolgreichen Angriff:
 - Anfragen verwerfen → Denial of Service
 - Anfragen mitlesen und/oder manipulieren → Man-in-the-Middle



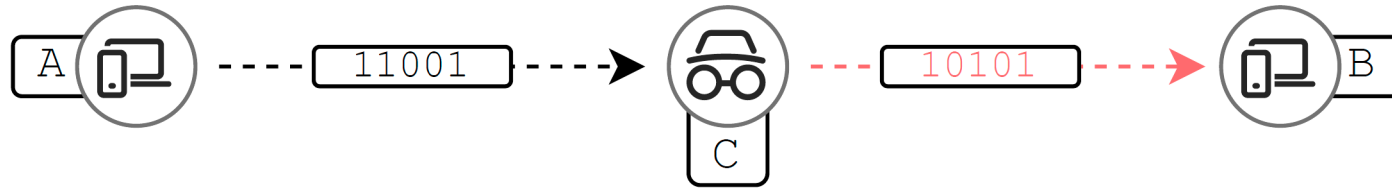
Denial of Service

- Annahme: Host A schickt Nachrichten an Host B. Ein Angreifer erlangt innerhalb des genutzten LANs Kontrolle über einen weiteren Host C , und gibt sich mittels ARP-Spoofing erfolgreich als Host B aus.
- Der Angreifer verwirft Nachrichten (ggf. selektiv) und blockiert Aktionen



Man-in-the-Middle

- Annahme: Host A schickt Nachrichten an Host B. Ein Angreifer erlangt innerhalb des genutzten LANs Kontrolle über einen weiteren Host C , und gibt sich mittels ARP-Spoofing erfolgreich als Host B aus.
- Der Angreifer **manipuliert Nachrichten** (ggf. selektiv) und leitet diese weiter



16.3 Sicherheitsmechanismen



Sicherheitsmechanismen

- Verschlüsselung (engl. encryption):
 - Daten sind nur für bestimmte Benutzer lesbar → Vertraulichkeit (confidentiality)
 - Integritätsprüfung auf unbefugte Änderungen → Integrität (integrity)s
 - Grundlage für unabstreitbare Operationen.
- Authentisierung (engl. authentication):
 - Überprüfung der Identität eines Benutzer, Client oder Servers, z.B. via Passwort
- Autorisierung (engl. authorization):
 - Überprüfung ob authentisierter Benutzer Zugriff hat.



Sicherheitsmechanismen

- Auditing:
 - Speichern von Benutzeraktivitäten (z.B. unerlaubte Zugriffsversuche),
 - Für die spätere Analyse.
- Wichtig für Sicherheitspolitiken:
 - Zusammenspiel aller Mechanismen ist entscheidend.
 - Eine Schwachstelle bringt alle Maßnahmen zu Fall.
 - Z.B. ein Pufferüberlauf führt u.U. zum Root-Zugriff auf gesamten Rechner.
→ Disziplin ist wichtig!

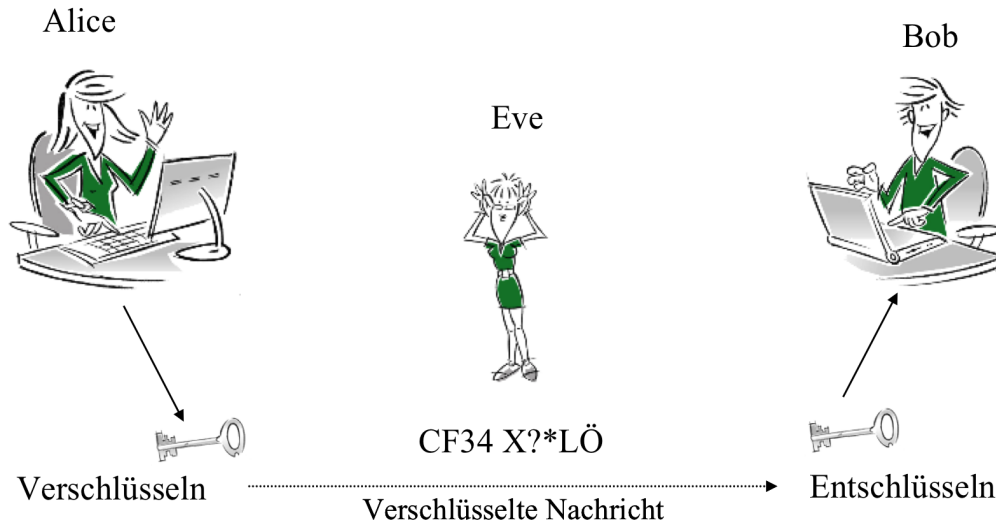


16.4 Symmetrische Verschlüsselung



Symmetrische Verschlüsselung

- Kommunikationspartner kennen beide einen geheimen Schlüssel
 - Data Encryption Standard (DES), IBM, 1975 (56 Bit Key, veraltet)
 - Advanced Encryption Standard (AES), (128 – 256 Bit Key)
 - Für Kommunikation & Dateien sowie Authentisierung, ...



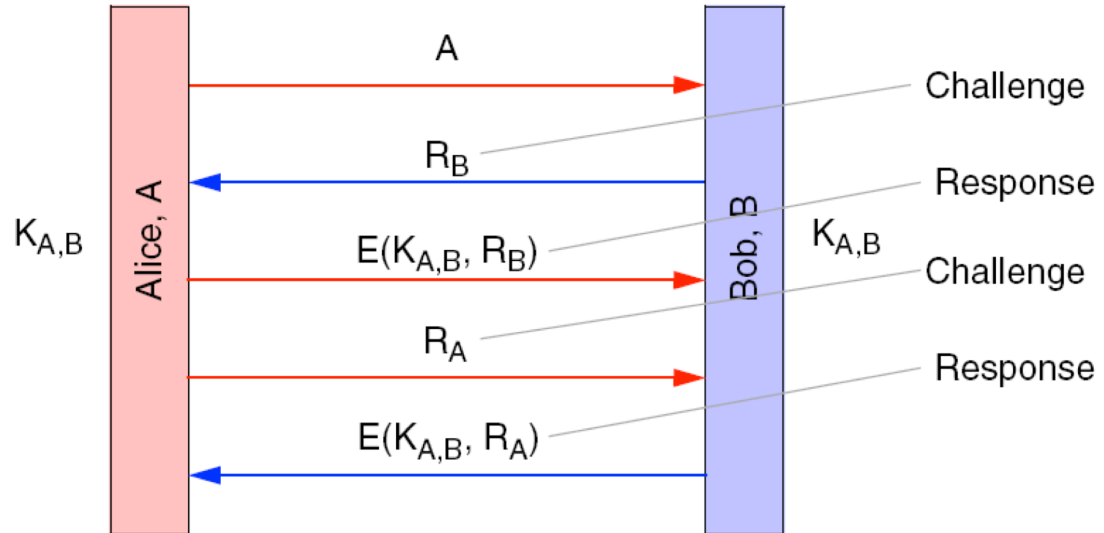
Symmetrische Verschlüsselung

- Funktionen:
 - Verschlüsselungsfunktion: E (encrypt): $E(K, T) \rightarrow C$
 - Entschlüsselungsfunktion: D (decrypt): $D(K, C) \rightarrow T$
 - K =Key , T =Text (zu verschlüsselnde Daten),
 $K_{A,B}$ geheimer Schlüssel von A und B.
- Forderungen:
 - Falls K unbekannt, soll es sehr aufwendig sein aus C das T zu ermitteln (Entschlüsselungsangriff).
 - Und es soll sehr aufwendig sein, aus T und C den Schlüssel K zu ermitteln (Klartextangriff).



Challenge-Response-Protokoll

- Authentisierung zwischen zwei Kommunikationspartnern:
 - R_B ist z.B. eine Zufallszahl.
 - Nach Nachricht 3 ist A für B eindeutig identifiziert.
 - A muss noch sicherstellen, dass wirklich B auf der anderen Seite ist (Nachrichten 4 + 5).



Challenge-Response-Protokoll

- Anschließender Datenaustausch über Verbindung:
 - Verschlüsselung der Daten mit $K_{A,B}$
 - Ausschluss von Verfälschung, sowie Einfügung und Verlust von Nachrichten.
- Wichtig: Beide Partner müssen $K_{A,B}$ kennen und sonst niemand.



Linux: gpg

message.json

Hexdump

```
00000000  7b 22 75 73 65 72 6e 61  6d 65 22 3a 22 68 65 69    |{"username":"hei|
00000010  6e 72 69 63 68 22 2c 22  70 61 73 73 77 6f 72 64    |nrich","password|
00000020  22 3a 22 68 65 69 6e 65  22 7d 0a                    |":"heine"}.|
```

Unter Linux kann das Tool `gpg` für die Verschlüsselung genutzt werden|



```
[student@uni]$ gpg --cipher-algo AES128 --batch --passphrase "abc123" -c message.json
```



- Verschlüsselter Text ist für einen Angreifer generell nutzlos

message.json.gpg

Hexdump

00000000	8c 0d 04 07 03 02 1c 40	79 03 77 99 e9 f4 f9 d2	@y.w.....
00000010	67 01 93 e4 09 67 c9 ad	f3 58 de 69 16 ee 82 0b	g....g...X.i....
00000020	b9 86 98 29 eb 31 2b a0	d4 8b d9 98 27 36 1f fe	...).1+.....'6..
00000030	10 ba fd 4f 47 0b f4 e0	2a e6 48 33 80 ff 35 4f	...0G...*.H3..50
00000040	82 dc 00 6c b2 51 63 32	54 b2 96 4c 84 43 a9 ee	...l.Qc2T..L.C..
00000050	62 28 35 c2 a4 c9 df 2d	b1 f7 09 d5 15 c3 40 3c	b(5....-.....@<
00000060	ec ed a4 cd d9 b1 f5 35	25 57 bc 81 15 67 9f e4	5%W...g...
00000070	e7 69 8a 9e 90 bd f9 2d		.i.....-

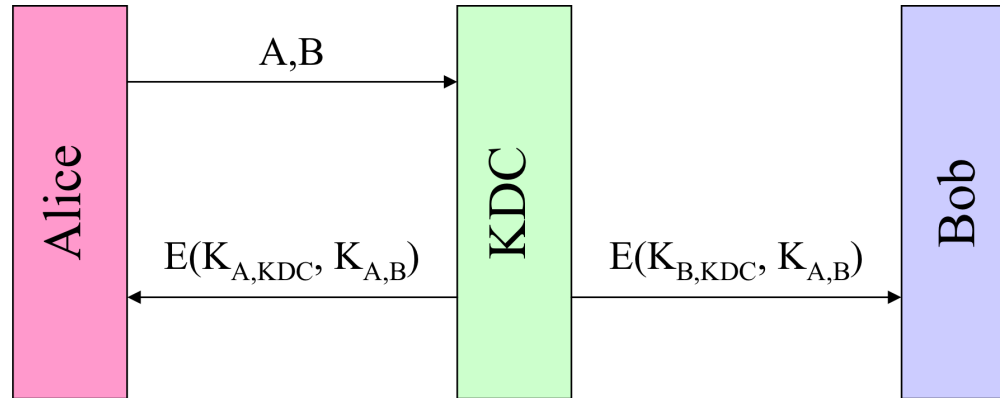
Schlüsselverwaltung

- Problem: Bei N Kommunikationspartnern sind $N \cdot (N-1)/2$ Schlüssel nötig.
- Lösung: Key Distribution Center (KDC)
 - Alle Teilnehmer haben einen geheimen Schlüssel, den nur das KDC kennt
 - Somit sind bei N Kommunikationspartnern nur noch N Schlüssel notwendig
 - Für die Kommunikation zwischen zwei Knoten generiert das KDC einen Sitzungsschlüssel.



Schlüsselverwaltung

- Verbindungsabbau zwischen A und B über KDC
 - KDC generiert Sitzungsschlüssel $K_{A,B}$ für einen Verbindungswunsch
 - $K_{A,B}$ wird durch KDC auf sicherem Weg (jeweils privater Key) zu A & B übertragen.
- Prinzip:



- Bem.: Sitzungsschlüssel wird nur für eine Sitzung verwendet und danach verworfen → schwerer zu knacken.

16.5 Asymmetrische Verschlüsselung



Asymmetrische Verfahren

- Statt einem Schlüssel wird hier ein Schlüsselpaar pro teilnehmendem Computer verwendet
 - K^+ = öffentlich bekannter Schlüssel (engl. public key)
 - K^- = geheimer privater Schlüssel (engl. private key)
- Funktionen:
 - Verschlüsselungsfunktion E (encrypt): $E(K^+, T) \rightarrow C$ $E(K^-, T) \rightarrow C'$
 - Entschlüsselungsfunktion D (decrypt): $D(K^-, C) \rightarrow T$ $D(K^+, C') \rightarrow T$
 - T: zu verschlüsselnde Daten.



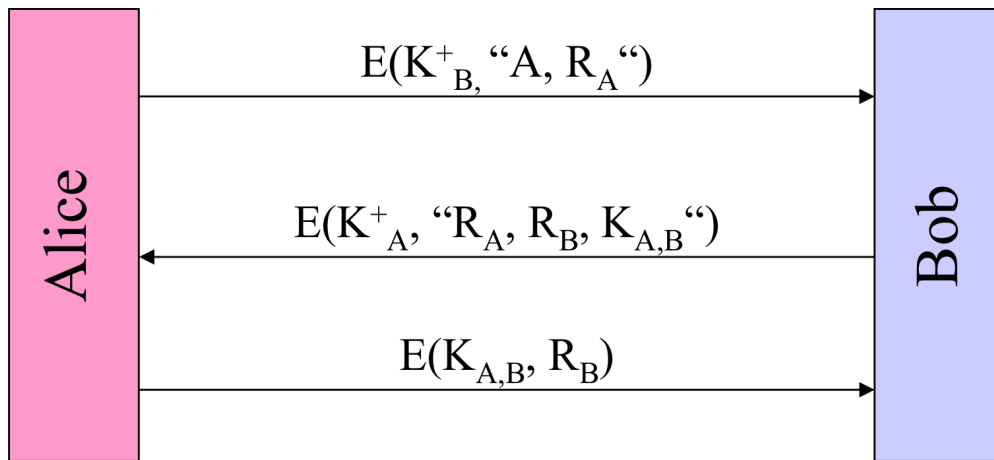
Asymmetrische Verfahren

- Forderungen an ein asymmetrisches Verfahren:
 - Wie bei symmetrischer Verschlüsselung,
 - Zusätzlich: aus bekanntem Schlüssel K^+ soll es nur mit hohem Aufwand möglich sein, den Schlüssel K^- zu berechnen und auch umgekehrt.
 - Zum Beispiel RSA
 - Benannt nach den Erfindern Rivest, Shamir und Adleman, 1977
- Vorteile asymmetrischer Verfahren:
 - Kein KDC erforderlich
 - Nur zwei Schlüssel für Kommunikation mit N Partnern
 - Aber Verbreitung des öffentlichen Schlüssels notwendig (siehe später)



Authentifizierung

- Anfrage kann nur von B entschlüsselt mit K_B^- werden.
- Antwort von B enthält Aufforderung R_B , damit A sich gegenüber B authentifiziert.
- Alice antwortet unter Verwendung des von B erzeugten Sitzungsschlüssels $K_{A,B}$ (nur sie kann diesen entschlüsseln)
- R_A und R_B sind Zufallszahlen, um das Einspielen alter Nachrichten zu erkennen.



16.6 Digitale Signatur



Hashfunktionen

- Hashfunktion: $H(T) \rightarrow D$
 - T: Text
 - D: Hash-Wert fester Länge (Digest)
- Forderungen an eine sichere Hashfunktionen
 - H: einfach und schnell zu berechnen
 - **Einwegeigenschaft:** Für einen gegebenen Wert D ist es nahezu unmöglich ein T zu finden, so dass gilt: $H(T) = D$
 - **Kollisionsresistenz:** Wenn H bekannt ist, ist nahezu unmöglich ein Paar (T, S) zu finden, so dass gilt: $H(T) = H(S)$



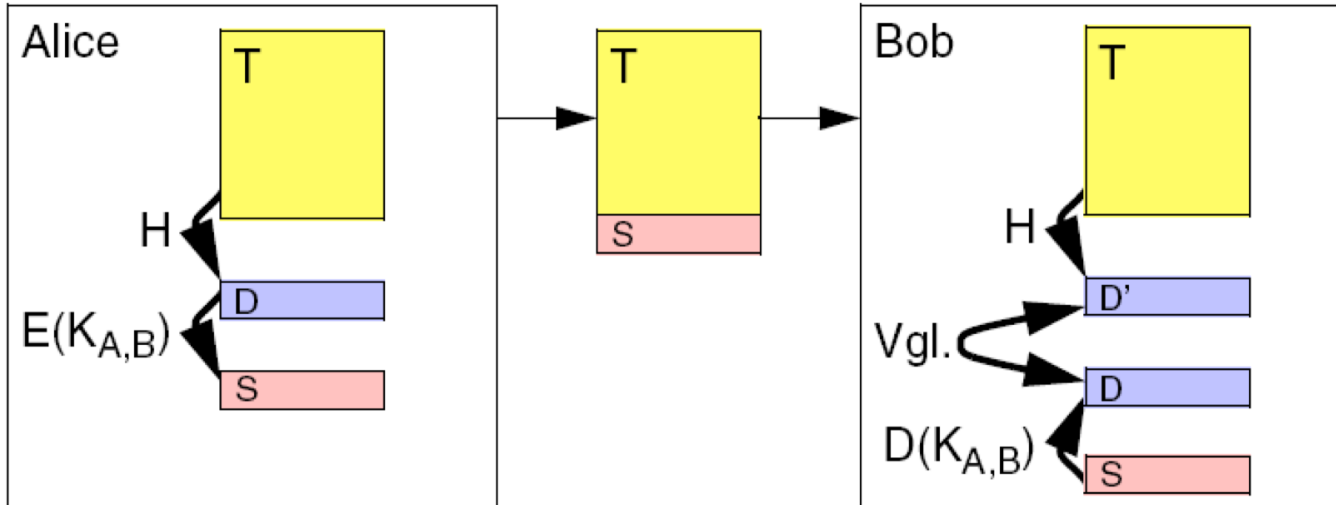
Digitale Signatur

- Digitale Signaturen mit sicheren Hash-Funktionen:
 - Nachricht wird im Klartext versendet
 - Digest / Hashwert wird verschlüsselt angehängt = Signatur
 - Damit kann niemand den Hashwert manipulieren
- Nutzen
 - **Integrität:** Anhand der Signatur kann der Empfänger prüfen, ob die Nachricht verändert wurde
 - **Authentisierung:** Mithilfe der Signatur kann der Empfänger feststellen, wer die Nachricht gesendet hat



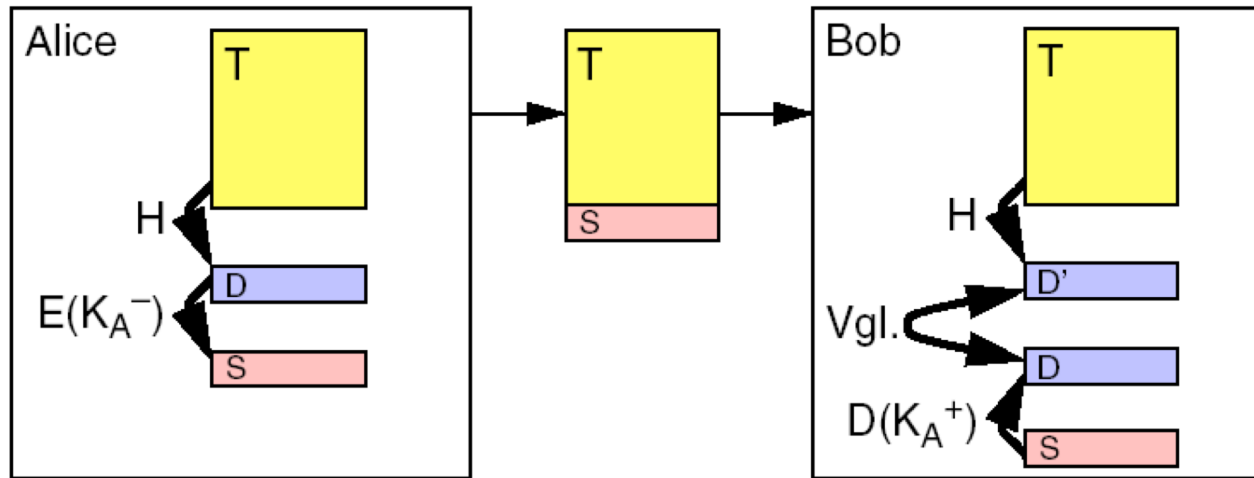
Digitale Signatur mit sym. Verschlüsselung

- D = Digest; S = Signatur (verschlüsselter Digest)
- $K_{A,B}$ Schlüssel für Kommunikation zw. Alice und Bob



Digitale Signatur mit asym. Verschlüsselung

- Digitale Signatur mit asymmetrischen Verfahren:
 - Signatur durch Verschlüsselung des Hashwerts einer Nachricht mit dem privaten Schlüssel,
 - Öffentlicher Schlüssel dient zum Überprüfen der Signatur.



Schlüsselzertifizierung

- Problem: Vertrauen in den öffentlichen Schlüssel:
 - Öffentlicher Schlüssel enthält Besitzerangabe (z.B. E-mail-Adresse),
 - Wann kann man den Angaben trauen?
 - wenn sie von vertrauenswürdigen Subjekten bestätigt werden
- Lösung: Zertifikat
 - = digital signierter öffentliche Schlüssel
 - Unterzeichner bestätigt die Authentizität des öffentlichen Schlüssel

Zerifikatbespiele

- Abstrakter Inhalt: Ich, der Aussteller, zertifiziere hiermit, dass ...
- ... das Fahrzeug mit der FIN WAZZGEXLAY28482 eine Betriebserlaubnis hat
- ... die Personalausweisnummer 3132552991 zu Max Mustermann gehört
- oder auch
- ... der Public Key X zum Host Y gehört
- Bem.: „zertifizieren“ bedeutet lediglich, dass der Aussteller etwas signiert/bestätigt

Schlüsselzertifizierung

- Beispiel: vertrauenswürdige Person Chuck signiert öffentlichen Key von Alice
 - Chuck zertifiziert den öffentlichen Schlüssel von Alice
 - Überzeugt sich, dass Alice privaten Schlüssel hat
 - Z.B. durch Challenge, Response und Verifizierung des Senders oder auf dem Postweg.
 - Bob vertraut Chuck:
 - Bob erhält öffentlichen Schlüssel von Alice,
 - Bob prüft diesen mithilfe des Zertifikates, also mithilfe der Signatur von Chuck (Integrität & Authentizität)
- Problem: Woher weiß Bob, dass Chucks öffentlicher Schlüssel zu Chuck gehört? → rekursives Problem.



Schlüsselzertifizierung - Lösung

- 1. Hierarchisch (z.B. Verisign, Trust Center):
 - Einer Wurzelautorität (Certification Authority) wird vertraut, deren K^+ ist wohlbekannt (Teil der Installation eines Betriebssystems)
 - Kette von Zertifikaten ergeben Vertrauensbasis.
- 2. Peer-to-Peer (z.B. PGP):
 - Mehrere digitale Signaturen für einen öffentlichen Schlüssel
 - Menge der vertrauenswürdigen Signaturen bestimmt letztliches Vertrauen.



Schlüsselzertifizierung - In der Praxis

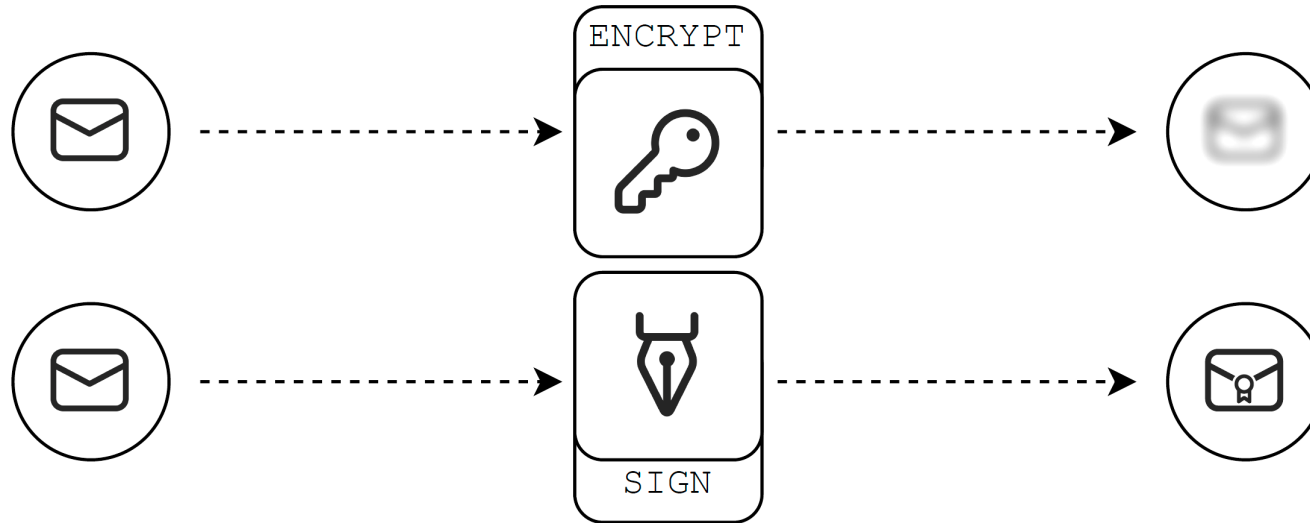
- Zertifikate nur für bestimmte Zeitdauer (Haltbarkeitsdatum)
- Zurückziehen von Zertifikaten:
 - Zertifizierungsstelle verwaltet Liste von zurückgezogenen Zertifikaten.
 - Empfänger muss die Signatur und die Liste prüfen.

16.7 Fallbeispiel: ARP Spoofing – Lösung



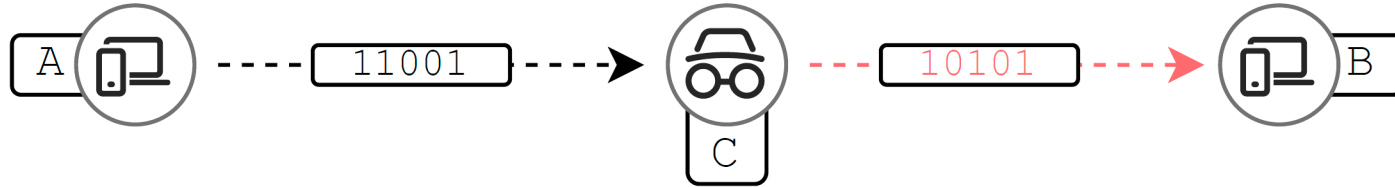
Schutzmaßnahmen

- Generell können zwei Maßnahmen gegen das (erfolgreiche) Abhören sowie die (erfolgreiche) Manipulation von Nachrichten ergriffen werden



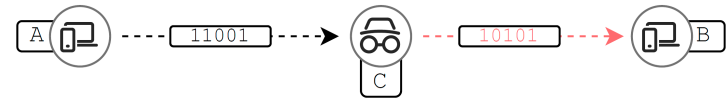
Man-in-the-Middle

- Auch wenn Daten verschlüsselt werden, kann ein Angreifer mittels ARP-Spoofing diese verändern ohne dass die beteiligten Parteien dies erkennen



- Aus Sicht der beteiligten Parteien ist die andere Seite nicht erreichbar oder die ankommenden Nachrichten "kaputt"
- **Lösung:** Server weisen beim Verbindungsaufbau zunächst ihre Identität nach

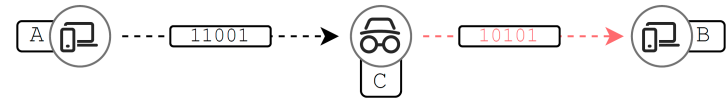
Man-in-the-Middle



- Was passiert, falls ein Angreifer auf einem dritten Host C während dem Schlüsselaustausch vorgibt das Ziel des Opfers zu sein (ARP-Spoofing)?
 1. Host A versucht eine Verbindung mit Host B aufzubauen
 2. Host C schickt eigenen Public-Key an Host B
 3. Host C verwirft Signaturen von Host A und signiert (veränderte) Nachrichten mit seinem eigenen Private-Key
 4. Host B verifiziert Nachrichten mit dem Public-Key von Host C und denkt sie stammen von Host A
- **Lösung:** Hosts bestätigen ihre Identität mithilfe von Zertifikaten



Man-in-the-Middle



- Durch den korrekten Einsatz von Zertifikaten, werden Man-in-the-Middle Angriffe praktisch unmöglich
- Beispiel: Host C gibt sich wieder als Host B aus:
 1. Host A versucht eine Verbindung mit Host B aufzubauen
 2. Host C fängt die Nachricht ab und antwortet Host A mit falschem Zertifikat
 3. Host A prüft Signatur und lehnt Zertifikat ab
 4. Host A erkennt, dass sich jemand anders als Host B ausgibt
- Hinweis: Host C kann weiterhin den Traffic umleiten, jedoch wird niemand mit ihm kommunizieren wollen, da sein Zertifikat ungültig ist Schutzmaßnahmen



16.8 Vergleich der Verschlüsselungsverfahren



Vergleich

- Symmetrische Verschlüsselung
 - Für große Datenmengen, da weniger Rechenaufwand
 - Es wird ein Shared-Key definiert (abc123 im vorhergehenden Beispiel)
 - Sender und Empfänger benutzen den selben Shared Key
 - Shared-Key muss auf sicherem Weg ausgetauscht werden
- Asymmetrische Verschlüsselung
 - Für kleine Datenmengen, da hoher Rechenaufwand
 - Empfänger generiert ein Schlüsselpaar (Public-Key & Private-Key)
 - Sender verschlüsselt Nachrichten mit Public-Key des Empfängers
⇒ Jeder darf Public-Key kennen
 - Empfänger entschlüsselt mit eigenem Private-Key
⇒ Private-Key wird nie übertragen / preisgegeben



Vergleich

- Was passiert, falls ein Angreifer während dem Schlüsselaustausch mithört?
- Symmetrisch
 - Angreifer kann den Shared-Key abfangen und anschließend die gesamte Kommunikation mitschneiden und entschlüsseln
- Asymmetrisch
 - Angreifer kann bloß den Public-Key “abfangen“ und die Kommunikation im Anschluss nicht entschlüsseln, da er keinen Private-Key hat
- Lösung: Shared-Key wird mit Public Key des Empfängers verschlüsselt und in dieser Form versendet ⇒ Angreifer “erkennt” nichts
 - Besser den Shared-Key periodisch neu erzeugen → „Sitzungsschlüssel“



16.9 Zusammenfassung



Zusammenfassung

- In ungesicherten Netzwerken kann alles abgehört und manipuliert werden
- Daten können symmetrisch und asymmetrisch verschlüsselt werden
⇒ Shared-Key vs. Public- und Private-Key
- Private Keys können zur Signierung von Daten genutzt werden
- Public Keys können zur Validierung von Signaturen genutzt werden
- Zertifikate sind durch vertrauenswürdige Instanzen signierte Daten
bzw. öffentliche Schlüssel

